

UNSW, School of Mathematics and Statistics

Profs Josef Dick and Kassem Mustapha

MATH3311/MATH5335

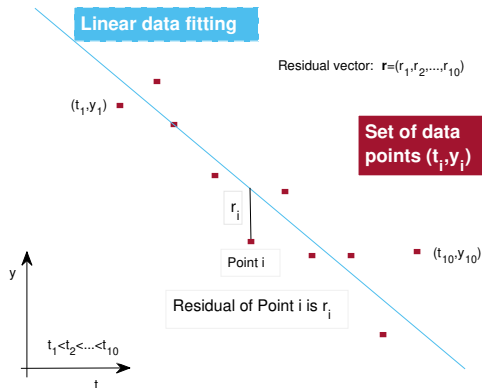
Topic 04 – Data fitting

1 Data Fitting – Parametric Methods

- Parametric functions
- Residuals
- Minimizing residual norms
- Least squares
- 1-norm
- Infinity norm
- Nonlinear models

Data fitting

Let us start with the following graphical illustration of simple linear data fitting.



How can we compute this line so that $\|\mathbf{r}\|_p$ is minimized for some $p \geq 1$? Here, $\mathbf{r}$ is the error vector (the residual), and we want to minimize its p -norm.

Common choices of p are 1, 2, and ∞ . In least-squares, $p = 2$, that is, we minimize $\sum_{i=1}^{10} r_i^2$.

Data fitting (continued)

- Data points

$$(t_i, y_i) \quad \text{for } i = 1, 2, \dots, m.$$

- Parametric function

$$\phi(\mathbf{x}; t) \quad \text{with } \mathbf{x} = (x_1, x_2, \dots, x_n)^T \in \mathbb{R}^n.$$

- In practice, $t_1 < t_2 < \dots < t_m$ and usually $t \in [t_1, t_m]$.
- Assumption:** $m \geq n$, where m is the number of data points and n is the number of parameters.
- Aim:** choose the parameters $\mathbf{x}$ so that

$$y_i \approx \phi(\mathbf{x}; t_i) \quad \text{for } i = 1, 2, \dots, m.$$

- Examples

- Linear fit:** $\phi(\mathbf{x}; t) = x_1 + x_2 t, \quad \mathbf{x} \in \mathbb{R}^2.$
- Quadratic fit:** $\phi(\mathbf{x}; t) = x_1 + x_2 t + x_3 t^2, \quad \mathbf{x} \in \mathbb{R}^3.$
- Trigonometric fit:** $\phi(\mathbf{x}; t) = x_1 + x_2 \cos(\pi t) + x_3 \sin(\pi t), \quad \mathbf{x} \in \mathbb{R}^3.$
- Gaussian fit:** $\phi(\mathbf{x}; t) = x_1 e^{-(t-x_2)^2/(2x_3^2)} + x_4 e^{-(t-x_5)^2/(2x_6^2)}, \quad \mathbf{x} \in \mathbb{R}^6.$

Residuals

- **Residual (column) vector** $\mathbf{r}(\mathbf{x}) = (r_1(\mathbf{x}), r_2(\mathbf{x}), \dots, r_m(\mathbf{x})) \in \mathbb{R}^m$ is defined by

$$r_i(\mathbf{x}) = \phi(\mathbf{x}; t_i) - y_i, \quad i = 1, \dots, m$$

- It measures the error in the approximation $y_i \approx \phi(\mathbf{x}; t_i)$ at the data points
- $\phi(\mathbf{x}; t)$ **interpolates** the data if

$$y_i = \phi(\mathbf{x}; t_i) \quad \text{for } i = 1, \dots, m$$

- Interpolation of the data is equivalent to $\mathbf{r}(\mathbf{x}) = \mathbf{0}$.
- Interpolation is only appropriate if the data are highly accurate.
- If (t_i, y_i) contain measurement errors, then we seek a small, rather than a zero, residual.
 - **Small** means a small p -norm $\|\mathbf{r}(\mathbf{x})\|_p$.

Minimizing residual norms

For data fitting, we choose an **estimator** $\mathbf{x}^*$ to minimize the p -norm of the residual:

$$\|\mathbf{r}(\mathbf{x}^*)\|_p = \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{r}(\mathbf{x})\|_p.$$

This problem is easiest to solve for $p=2$, in which case we obtain a **least-squares approximation**.

It is also simplest to assume that $\phi(\mathbf{x}; t)$ is **linear** in $\mathbf{x}$, so that

$$\phi(\mathbf{x}; t) = \sum_{j=1}^n x_j \phi_j(t)$$

for some **basis functions** $\phi_1(t)$, $\phi_2(t)$, $\dots$, $\phi_n(t)$.

Linear problems

- **Linear problem:** $\phi(\mathbf{x}; t) = \sum_{j=1}^n x_j \phi_j(t)$.
- Recall that $r_i(\mathbf{x}) = \phi(\mathbf{x}; t_i) - y_i$ for $i = 1, \dots, m$, and thus the residual vector is

$$\mathbf{r}(\mathbf{x}) = \begin{bmatrix} r_1(\mathbf{x}) \\ r_2(\mathbf{x}) \\ \vdots \\ r_m(\mathbf{x}) \end{bmatrix} = \underbrace{\begin{bmatrix} \phi_1(t_1) & \phi_2(t_1) & \cdots & \phi_n(t_1) \\ \phi_1(t_2) & \phi_2(t_2) & \cdots & \phi_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(t_m) & \phi_2(t_m) & \cdots & \phi_n(t_m) \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}_{\mathbf{x}} - \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}}_{\mathbf{y}}$$

Thus, $\mathbf{r}(\mathbf{x}) = A\mathbf{x} - \mathbf{y}$ where $A = [\phi_j(t_i)] \in \mathbb{R}^{m \times n}$.

How do we find $\mathbf{x}^*$ such that $\|\mathbf{r}(\mathbf{x}^*)\|_2 = \min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{y}\|_2$?

- **Example 1:** $\phi(x_1, x_2; t) = x_1 + x_2 t$, linear fit in t , so

$$\mathbf{r}(\mathbf{x}) = \begin{bmatrix} 1 & t_1 \\ \vdots & \vdots \\ 1 & t_m \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}$$

Linear problems (continued)

- **Example 2:** $\phi(\mathbf{x}; t) = x_1 + x_2 e^{rt}$, r known, $m = 4$

$$\mathbf{r}(\mathbf{x}) = \underbrace{\begin{bmatrix} 1 & e^{rt_1} \\ 1 & e^{rt_2} \\ 1 & e^{rt_3} \\ 1 & e^{rt_4} \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_{\mathbf{x}} - \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}}_{\mathbf{y}}$$

- **Example 3:** $\phi(x_1, x_2; t) = x_1 + x_2 t + x_3 e^t + x_4 \sin(t)$ and $m = 6$, so

$$\mathbf{r}(\mathbf{x}) = \underbrace{\begin{bmatrix} 1 & t_1 & e^{t_1} & \sin(t_1) \\ 1 & t_2 & e^{t_2} & \sin(t_2) \\ 1 & t_3 & e^{t_3} & \sin(t_3) \\ 1 & t_4 & e^{t_4} & \sin(t_4) \\ 1 & t_5 & e^{t_5} & \sin(t_5) \\ 1 & t_6 & e^{t_6} & \sin(t_6) \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_{\mathbf{x}} - \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \end{bmatrix}}_{\mathbf{y}}$$

Linear least-squares – Normal equations

For $\phi(\mathbf{x}; t) = \sum_{j=1}^n x_j \phi_j(t)$, a solution $\mathbf{x}^*$ of the normal equations¹

$$A^T A \mathbf{x} = A^T \mathbf{y} \quad \text{minimizes} \quad \|\mathbf{r}(\mathbf{x})\|_2.$$

That is, $\|\mathbf{r}(\mathbf{x}^*)\|_2 = \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{r}(\mathbf{x})\|_2 = \min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{y}\|_2$.

Proof: let $\mathbf{x} = \mathbf{x}^* + \mathbf{u}$, where $\mathbf{u} \in \mathbb{R}^n$. Claim: $\|\mathbf{r}(\mathbf{x})\|_2 \geq \|\mathbf{r}(\mathbf{x}^*)\|_2$.

$$\mathbf{r}(\mathbf{x}) = \mathbf{r}(\mathbf{x}^* + \mathbf{u}) = A(\mathbf{x}^* + \mathbf{u}) - \mathbf{y} = \mathbf{r}(\mathbf{x}^*) + A\mathbf{u}.$$

Thus,

$$\|\mathbf{r}(\mathbf{x})\|_2^2 = (\mathbf{r}(\mathbf{x}^*) + A\mathbf{u})^T (\mathbf{r}(\mathbf{x}^*) + A\mathbf{u}) = \|\mathbf{r}(\mathbf{x}^*)\|_2^2 + 2(A\mathbf{u})^T \mathbf{r}(\mathbf{x}^*) + \|A\mathbf{u}\|_2^2.$$

Because $\mathbf{x}^*$ satisfies the normal equations,

$$(A\mathbf{u})^T \mathbf{r}(\mathbf{x}^*) = \mathbf{u}^T A^T \mathbf{r}(\mathbf{x}^*) = \mathbf{u}^T (A^T A \mathbf{x}^* - A^T \mathbf{y}) = 0, \text{ so}$$

$$\|\mathbf{r}(\mathbf{x})\|_2^2 = \|\mathbf{r}(\mathbf{x}^*)\|_2^2 + \|A\mathbf{u}\|_2^2 \geq \|\mathbf{r}(\mathbf{x}^*)\|_2^2. \quad \square$$

¹They are called the normal equations because the residual is orthogonal to the column space of A .

Linear least-squares – Normal equations (continued)

- **Example:** Use the normal equations to compute $\mathbf{x}^*$ when $\phi(\mathbf{x}; t) = x_1 + x_2 t$.

- Recall that

$$\mathbf{r}(\mathbf{x}) = \underbrace{\begin{bmatrix} 1 & t_1 \\ \vdots & \vdots \\ 1 & t_m \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_{\mathbf{x}} - \underbrace{\begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}}_{\mathbf{y}}, \quad A^T = \begin{bmatrix} 1 & 1 & \dots & 1 \\ t_1 & t_2 & \dots & t_m \end{bmatrix}$$

- $\mathbf{x}^*$ is the solution of the normal equation $B\mathbf{x} = \mathbf{b}$ where $B = A^T A$ and $\mathbf{b} = A^T \mathbf{y}$. Thus,

$$\begin{bmatrix} m & \sum_{i=1}^m t_i \\ \sum_{i=1}^m t_i & \sum_{i=1}^m t_i^2 \end{bmatrix} \begin{bmatrix} x_1^* \\ x_2^* \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m y_i \\ \sum_{i=1}^m t_i y_i \end{bmatrix}.$$

- $\mathbf{v}^T B \mathbf{v} = (A\mathbf{v})^T A\mathbf{v} = \|A\mathbf{v}\|_2^2 \geq 0$, so the symmetric matrix B is positive semidefinite. Hence, if B has a full rank, then it is positive-definite.
- If B is positive-definite, use **Cholesky factorization** to solve $B\mathbf{x} = \mathbf{b}$.
- Forming $B = A^T A$ is **numerically bad**, since $\kappa_2(B) = (\kappa_2(A))^2$.

Linear least-squares – QR factorization

- It is best to compute $\mathbf{x}^*$ via the QR-factorization with column pivoting

$$AP = Q \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix}, \quad Q = [Y \quad Z],$$

where $A \in \mathbb{R}^{m \times n}$ with $m \geq n$, $Q \in \mathbb{R}^{m \times m}$ orthogonal, $R \in \mathbb{R}^{n \times n}$ upper triangular, $P \in \mathbb{R}^{n \times n}$ a permutation matrix, $Y \in \mathbb{R}^{m \times n}$, and $Z \in \mathbb{R}^{m \times (m-n)}$.

- The residual satisfies

$$\begin{aligned} \mathbf{r}(\mathbf{x}) &= A\mathbf{x} - \mathbf{y} = APP^T\mathbf{x} - \mathbf{y} = Q \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix} P^T\mathbf{x} - QQ^T\mathbf{y} \\ &= Q \left(\begin{bmatrix} RP^T\mathbf{x} \\ \mathbf{0} \end{bmatrix} - \begin{bmatrix} Y^T \\ Z^T \end{bmatrix} \mathbf{y} \right) = Q \begin{bmatrix} RP^T\mathbf{x} - Y^T\mathbf{y} \\ -Z^T\mathbf{y} \end{bmatrix}. \end{aligned}$$

- Since Q is orthogonal, $\|\mathbf{r}(\mathbf{x})\|_2^2 = \mathbf{r}^T(\mathbf{x})\mathbf{r}(\mathbf{x}) =$

$$\begin{bmatrix} RP^T\mathbf{x} - Y^T\mathbf{y} \\ -Z^T\mathbf{y} \end{bmatrix}^T Q^T Q \begin{bmatrix} RP^T\mathbf{x} - Y^T\mathbf{y} \\ -Z^T\mathbf{y} \end{bmatrix} = \|RP^T\mathbf{x} - Y^T\mathbf{y}\|_2^2 + \|Z^T\mathbf{y}\|_2^2.$$

- Therefore, $\mathbf{x}^*$ minimizes $\|\mathbf{r}(\mathbf{x})\|_2$ iff $\mathbf{x}^*$ solves the system: $R(P^T\mathbf{x}) = Y^T\mathbf{y}$.

Note: MATLAB `A \ y` computes the above $\mathbf{x}^*$ directly.

Linear least-squares – SVD

- Instead of the QR-factorization, we can use the SVD:

$$A = U \begin{bmatrix} \Sigma \\ \mathbf{0} \end{bmatrix} V^T, \quad U = [Y \quad Z]$$

where $A \in \mathbb{R}^{m \times n}$ with $m \geq n$, $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_n)$, $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal, and $Y \in \mathbb{R}^{m \times n}$.

- The residual is

$$\mathbf{r}(\mathbf{x}) = A\mathbf{x} - \mathbf{y} = U \begin{bmatrix} \Sigma \\ \mathbf{0} \end{bmatrix} V^T \mathbf{x} - U U^T \mathbf{y} = U \begin{bmatrix} \Sigma V^T \mathbf{x} - Y^T \mathbf{y} \\ -Z^T \mathbf{y} \end{bmatrix}.$$

- Since U is orthogonal, $\|\mathbf{r}(\mathbf{x})\|_2^2 = \mathbf{r}(\mathbf{x})^T \mathbf{r}(\mathbf{x}) =$

$$\begin{bmatrix} \Sigma V^T \mathbf{x} - Y^T \mathbf{y} \\ -Z^T \mathbf{y} \end{bmatrix}^T U^T U \begin{bmatrix} \Sigma V^T \mathbf{x} - Y^T \mathbf{y} \\ -Z^T \mathbf{y} \end{bmatrix} = \|\Sigma V^T \mathbf{x} - Y^T \mathbf{y}\|_2^2 + \|Z^T \mathbf{y}\|_2^2.$$

- $\mathbf{x}^*$ minimizes $\|\mathbf{r}(\mathbf{x})\|_2$ iff $\mathbf{x}^*$ solves the linear system: $\Sigma V^T \mathbf{x} = Y^T \mathbf{y}$.

Linear least-squares – SVD (continued)

- If A has full column rank, then

$$\mathbf{x}^* = V \Sigma^{-1} Y^T \mathbf{y}, \quad \Sigma^{-1} = \text{diag}(\sigma_1^{-1}, \dots, \sigma_n^{-1}).$$

- If $\text{rank}(A) = r < n$ (rank-deficient), then the least-squares solution is not unique.

In this case, and as we explain next, we choose a **minimum-norm** solution. Decompose the diagonal matrix Σ as:

$$\Sigma = \begin{bmatrix} \Sigma_r & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix},$$

where $\Sigma_r = \text{diag}(\sigma_1, \dots, \sigma_r)$. For later use, let

$$\Sigma^+ = \begin{bmatrix} \Sigma_r^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix},$$

where $\Sigma_r^{-1} = \text{diag}(1/\sigma_1, \dots, 1/\sigma_r)$ is the inverse of Σ .

Linear least-squares – SVD (continued)

Recall that $\|\mathbf{r}(\mathbf{x})\|_2^2 = \|\Sigma V^T \mathbf{x} - Y^T \mathbf{y}\|_2^2 + \|Z^T \mathbf{y}\|_2^2$. Let

$$V^T \mathbf{x} = \begin{bmatrix} \mathbf{w}_1 \\ \mathbf{w}_2 \end{bmatrix}, \quad Y^T \mathbf{y} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix}, \quad \|\mathbf{x}\|_2^2 = \|V^T \mathbf{x}\|_2^2 = \|\mathbf{w}_1\|_2^2 + \|\mathbf{w}_2\|_2^2,$$

where $\mathbf{w}_1, \mathbf{b}_1 \in \mathbb{R}^r$ and $\mathbf{w}_2, \mathbf{b}_2 \in \mathbb{R}^{n-r}$. Then,

$$\Sigma V^T \mathbf{x} - Y^T \mathbf{y} = \begin{bmatrix} \Sigma_r & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{w}_1 \\ \mathbf{w}_2 \end{bmatrix} - \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix} = \begin{bmatrix} \Sigma_r \mathbf{w}_1 - \mathbf{b}_1 \\ -\mathbf{b}_2 \end{bmatrix}.$$

Hence,

$$\|\Sigma V^T \mathbf{x} - Y^T \mathbf{y}\|_2^2 = \|\Sigma_r \mathbf{w}_1 - \mathbf{b}_1\|_2^2 + \|\mathbf{b}_2\|_2^2.$$

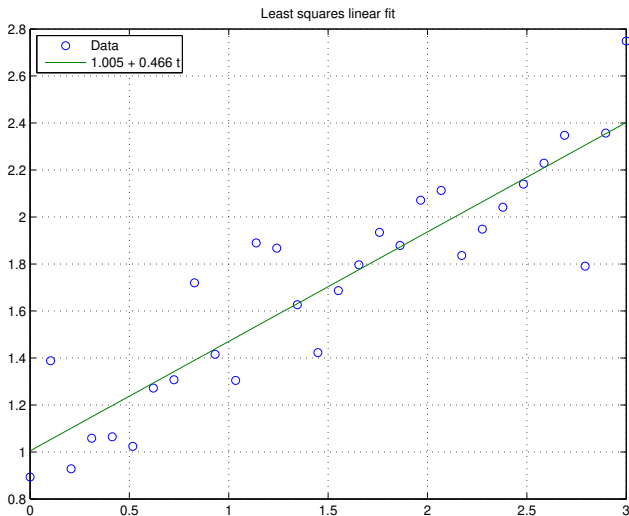
Therefore, $\mathbf{x}^*$ minimizes $\|\mathbf{r}(\mathbf{x})\|_2$ iff $\Sigma_r \mathbf{w}_1 = \mathbf{b}_1$. We choose $\mathbf{w}_2 = \mathbf{0}$ so that $\|\mathbf{x}\|_2$ is minimized, and thus,

$$V^T \mathbf{x}^* = \begin{bmatrix} \Sigma_r^{-1} \mathbf{b}_1 \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \Sigma_r^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix} = \Sigma^+ Y^T \mathbf{y} \implies \mathbf{x}^* = V \Sigma^+ Y^T \mathbf{y}.$$

- $\mathbf{x}^*$ minimizes $\|\mathbf{x}\|_2$ among all minimizers of $\|\mathbf{r}(\mathbf{x})\|_2$.
- Python `lstsq(A,y)[0]` computes the above $\mathbf{x}^*$ directly.

Example: Linear least-squares fit

MATLAB `linfit.m`, Python `linfit.py`



Choice of norm, iid: independent and identically distributed

- Minimizing the 2-norm of the residual,

$$\|\mathbf{r}(\mathbf{x})\|_2 = \sqrt{\sum_{i=1}^m (\phi(\mathbf{x}; t_i) - y_i)^2}$$

is appropriate if the measurement errors in the data y_i are independent, normally distributed random variables **with the same variance σ^2** :

$$y_i = \phi(\mathbf{x}; t_i) + \epsilon_i, \quad \epsilon_i \text{ are iid } \mathcal{N}(0, \sigma^2).$$

- A notable advantage of linear least-squares is that the fitted solution $\mathbf{x}^*$ depends **linearly** on the data $\mathbf{y}$.
- If the data include **outliers**, then we often obtain a more robust fit by minimizing the 1-norm of the residual,

$$\|\mathbf{r}(\mathbf{x})\|_1 = \sum_{i=1}^m |\phi(\mathbf{x}; t_i) - y_i|.$$

Linear 1-norm estimation: no explicit solution

- We can compute the estimator $\mathbf{x}^*$ that minimizes the residual in the 1-norm by solving the **optimization problem**

$$\begin{aligned} & \text{Minimize} && \sum_{i=1}^m v_i \\ & \text{Subject to} && -v_i \leq r_i(\mathbf{x}) \leq v_i, \quad i = 1, \dots, m, \\ & && \mathbf{v} \geq \mathbf{0} \text{ (i.e., } v_i \geq 0, \text{ } i = 1, \dots, m), \mathbf{x} \in \mathbb{R}^n, \mathbf{v} \in \mathbb{R}^m. \end{aligned} \quad (1)$$

- $v_i \geq |r_i(\mathbf{x})| \iff v_i \geq r_i(\mathbf{x}) \text{ and } v_i \geq -r_i(\mathbf{x}).$
- At the solution, $v_i = |r_i(\mathbf{x}^*)|$, so $\|\mathbf{r}(\mathbf{x}^*)\|_1 = \sum_{i=1}^m v_i.$
- For **linear** problems, $\mathbf{r}(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{y}$, the optimization problem in (1) is a **linear programming problem**.
 - MATLAB `linprog`, Python `scipy.optimize.linprog` solves linear programming problems. MATLAB `l1p.m` and Python `l1p.py` use `linprog` to minimize $\|\mathbf{A}\mathbf{x} - \mathbf{y}\|_1$

Linear ∞ -norm estimation

- If the data (t_i, y_i) are **very accurate**, then we usually want to minimize the ∞ -norm of the residual,

$$\|\mathbf{r}(\mathbf{x})\|_{\infty} = \max_{1 \leq i \leq m} |\phi(\mathbf{x}; t_i) - y_i|.$$

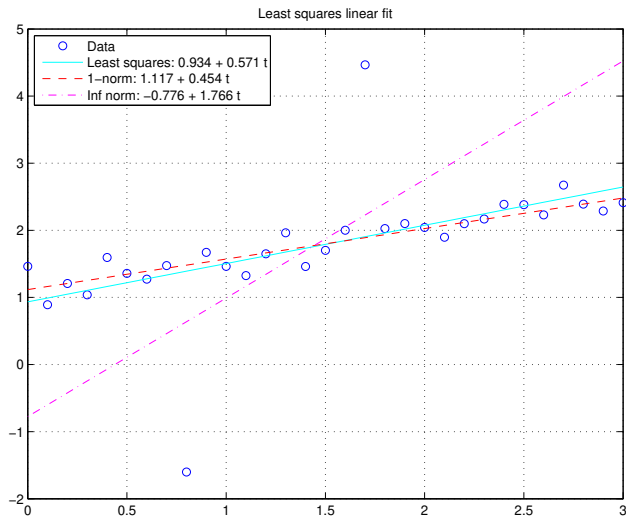
- Find the estimator $\mathbf{x}^*$ by solving the optimization problem

$$\begin{aligned} &\text{Minimize} && v \\ &\text{Subject to} && -v \leq r_i(\mathbf{x}) \leq v, \quad i = 1, \dots, m, \\ & && v \geq 0, \quad \mathbf{x} \in \mathbb{R}^n, \quad v \in \mathbb{R}. \end{aligned} \tag{2}$$

- $v \geq |r_i(\mathbf{x})|$ for all $i = 1, \dots, m \implies v \geq \max_{1 \leq i \leq m} |r_i(\mathbf{x})|$.
- At the solution, $v^* = \|\mathbf{r}(\mathbf{x}^*)\|_{\infty}$.
- For **Linear** problems $\mathbf{r}(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{y}$, the optimization problem in (2) is a **linear programming problem**.
 - MATLAB `linprog`, Python `scipy.optimize.linprog` solve linear programming problems. MATLAB `lilp.m` and Python `lilp.py` use `linprog` to minimize $\|\mathbf{A}\mathbf{x} - \mathbf{y}\|_{\infty}$.

Example: least-squares, 1-norm, and ∞ -norm estimates

MATLAB `linfit1.m`, Python `linfit1.py`



Nonlinear models - Transformation to linearize the model

Finding an estimator $\mathbf{x}^*$ is difficult if $\phi(\mathbf{x}; t)$ is nonlinear in $\mathbf{x}$. Sometimes a **change of variables** helps.

Example: Consider the nonlinear model

$$y_i \approx \phi(\mathbf{x}; t_i) \quad \text{for} \quad \phi(\mathbf{x}; t) = x_1 \prod_{j=2}^n (t - \beta_j)^{x_j},$$

where $\beta_2, \dots, \beta_n$ are known, $t > \max_{2 \leq j \leq n} \beta_j$, $x_1 > 0$.

- Exponentials, products, powers **suggest** a log transformation:

$$\log \phi(\mathbf{x}; t) = \log x_1 + \sum_{j=2}^n x_j \log(t - \beta_j).$$

- New variables** $\mathbf{z} = [\log x_1, x_2, \dots, x_n]^T$ and **basis functions**

$$\psi_j(t) = \begin{cases} 1, & j = 1, \\ \log(t - \beta_j) & 2 \leq j \leq n, \end{cases} \implies \psi(\mathbf{z}; t) = \sum_{j=1}^n z_j \psi_j(t).$$

- Linear model** $\log y_i \approx \psi(\mathbf{z}; t_i)$, assuming $y_i > 0$.